Laboratório Prolog · Aula 13

Matheus Pintor Fernandes Ferreira · RA 24000551-2 · Turma ESOFT6S-C
Gerado em 19/10/2026, 20:25:52 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) encontrará L = scheme. paradigma(java, logico) retornará false, pois o único paradigma de java registrado é orientado_a_objetos.

Saída do ola.pl

```
SAÍDA PREVISTA (não executada no OneCompiler):
?- linguagem(A). → prolog; scheme; java
?- paradigma(prolog,A). → logico
?- paradigma(A,funcional). → scheme
?- paradigma(java,logico). → false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

A primeira pergunta qual é o paradigma associado a prolog e descobre P = logico. A segunda apenas verifica se o fato paradigma(prolog, logico) pode ser provado; a resposta é verdadeira.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Significa que o programa não consegue provar essa afirmação com os fatos disponíveis. Pela hipótese do mundo fechado, consultas não demonstráveis falham; isso não é uma análise completa de todas as características da linguagem Java.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) tem 7 respostas: lucia, marta, tereza, joana, bia, ana e dora. Para (mae(M,F),mulher(F)): (lucia,tereza), (marta,joana), (tereza,bia) e (joana,ana).

Saída do familia.pl

```
SAÍDA PREVISTA (não executada no OneCompiler):
mae(joana,jake): verdadeiro; mae(jake,joana): false; pai(maria,jake): false.
mae(joana,F): jake, ana; pai(P,jake): jorge.
mulher(X): lucia, marta, tereza, joana, bia, ana, dora.
mae(joana,X),homem(X): jake.
pai(vern,Y),mulher(Y): joana.
mae(M,F),mulher(F): (lucia,tereza), (marta,joana), (tereza,bia), (joana,ana).
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Pergunta quais pessoas são mães de filhas identificadas como mulheres na base. Há 4 combinações: Mae=lucia/Filha=tereza; marta/joana; tereza/bia; joana/ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: aquilo que não está registrado nem pode ser deduzido é tratado como falso. Em situações reais, ausência de informação pode significar apenas cadastro incompleto, não falsidade do fato.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

lucia, marta, tereza, joana, bia, ana e dora. Essa sequência é a ordem em que os fatos mulher/1 aparecem no programa, examinados de cima para baixo pelo mecanismo de busca.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X,jake) retorna X = paulo, irmão de joana. primo(leo,X) retorna X = jake e X = ana, pois paulo (pai de leo) e joana (mãe de jake e ana) são irmãos.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham f(X,X)=f(a,b), [A|B]=[] e 2+3=5. Unifica f(X,X)=f(a,Y), produzindo X=a e Y=a. O sinal = compara a estrutura dos termos, sem calcular a soma.

Saídas de regras.pl e unificacao.pl

```
SAÍDAS PREVISTAS (não executadas):
regras.pl: avo(A,jake): marta, vern; avo(osvaldo,N): joana, paulo, bia; irmao(jake,Y): ana;
tio(T,jake): paulo; primo(leo,P): jake, ana; ancestral(jake,_): false.
unificacao.pl: a=a unifica; a=b NÃO; f(X,X)=f(a,b) NÃO; f(X,X)=f(a,Y) SIM (X=a,Y=a); [H|T]=
[1,2,3] SIM; [H|T]=[] NÃO; 2+3=5 NÃO.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

O caso base ancestral(X,Y) :- progenitor(X,Y) identifica um pai ou uma mãe diretamente. O caso recursivo ancestral(X,Y) :- progenitor(X,Z), ancestral(Z,Y) alcança gerações adicionais; ambos são necessários para cobrir parentesco direto e indireto.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a regra admite irmao(jake,jake), pois uma pessoa compartilha pai e mãe consigo própria. A desigualdade impede que o próprio indivíduo seja identificado como seu irmão.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X,X)=f(a,b), X teria de valer simultaneamente a e b, impossível. Em f(X,X)=f(a,Y), basta X=a e Y=a para que as duas estruturas coincidam.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Prolog, a relação avo/2 permite perguntar tanto pelos avós de alguém quanto pelos netos de um avô, mudando as variáveis da consulta. Uma função Java ou Scheme tradicional recebe entradas e calcula um resultado em direção definida; para buscar o inverso, normalmente é necessário escrever lógica adicional.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

Há quatro decomposições: X=[], Y=[1,2,3]; X=[1], Y=[2,3]; X=[1,2], Y=[3]; X=[1,2,3], Y=[].

Saída do listas.pl

```
SAÍDA PREVISTA (não executada):
membro(b,[a,b,c]): verdadeiro; membro(z,[a,b,c]): false; membro(X,[a,b,c]): a,b,c.
```

```
tamanho([a,b,c],N): N=3; tamanho([],N): N=0.
concatena([1,2],[3,4],R): R=[1,2,3,4].
concatena(X,[3],[1,2,3]): X=[1,2].
concatena([1],Y,[1,2,3]): Y=[2,3].
concatena(X,Y,[1,2,3]): quatro divisões da lista.
inverte([1,2,3],R): R=[3,2,1].
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([],0). Passo: tamanho([_|Cauda],N) :- tamanho(Cauda,M), N is M+1. Para [a,b,c], ficam pendentes três incrementos (+1), que são calculados quando a chamada com lista vazia retorna 0.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque a lista [1,2,3] pode ser dividida em prefixo e sufixo em quatro posições, inclusive nas extremidades. O concatena/3 pode descobrir listas desconhecidas. A função anexar de Scheme normalmente concatena duas listas recebidas, mas não enumera automaticamente todas as possíveis decomposições de uma lista-resultante.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

A unificação encontra o primeiro elemento; o backtracking volta à alternativa recursiva e examina a cauda, produzindo a, b e c sem um laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2+3 associa X ao termo simbólico +(2,3), sem calcular. X is 2+3 calcula e resulta em X=5. fatorial(N,120), com N não instanciado, provoca erro de instanciação na comparação N>0.

Saída do aritmetica.pl

```
SAÍDA PREVISTA (não executada):
X = 2+3: termo 2+3; X is 2+3: 5; 5 =:= 2+3: verdadeiro; 2+3 = 5: false.
7 mod 3 = 1; 7 // 2 = 3; 2 ^ 10 = 1024.
fatorial(5,F): F=120; fatorial(0,F): F=1; fib(10,F): F=55; soma([1,2,3,4],S): S=10.
fatorial(N,120) e X is Y+1 com variável livre: erros de instanciação.
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= faz unificação estrutural: X = 2+3 vincula X ao termo 2+3. is avalia o lado direito: X is 2+3 resulta em X=5. =:= compara valores numéricos de expressões já avaliáveis: 5 =:= 2+3 é verdadeiro, mas 2+3 = 5 falha.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

A regra de fatorial usa N>0 e N1 is N-1, que exigem N conhecido. Já concatena é definida por unificação e padrões de listas, permitindo descobrir valores desconhecidos nos seus argumentos.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam recursão e um caso base 0! = 1, mais n! = n × (n-1)!. No Haskell, fact é uma função que avalia entrada para obter saída; no Prolog, fatorial/2 é um predicado relacional, porém o uso de > e is torna a implementação aritmética dependente de N conhecido.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1(X) encontrará osvaldo, paulo e jake; solteiro2(X) dará false, porque tenta negar casado(X) com X livre. maximo2(5,3,M) retorna duas respostas (M=5 e M=3). maximo3(5,3,3) retorna true indevidamente.

Saída de controle.pl e mensagem de ancestral_ruim

```
SAÍDA PREVISTA (não executada):
mesmo_gosto(X,Y): (ana,caio) e (caio,ana); tentativa (ana,ana) falha pela desigualdade.
solteiro1(X): osvaldo, paulo, jake; solteiro2(X): false.
maximo1(5,3,M): M=5; maximo2(5,3,M): M=5 e M=3; maximo3(5,3,M): M=5; maximo1(5,3,3): false;
maximo3(5,3,3): verdadeiro (defeito lógico).
laco_infinito.pl: ancestral_bom(osvaldo,jake): sim; ancestral_ruim(osvaldo,jake): erro de
estouro de pilha/limite de recursão (mensagem exata depende do ambiente).
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com três cores distintas, há 24 colorações válidas; com apenas duas cores, nenhuma, pois PR, SP e MS formam um triângulo de fronteiras.

Saída do mapa.pl

```
SAÍDA PREVISTA (contagem calculada a partir das restrições, não executada no OneCompiler):
3 cores: 24 colorações.
2 cores: false / 0 colorações.
4 cores: 1728 colorações.
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O mecanismo tenta pares em ordem. Em ana/ana, a meta X \== Y falha por serem a mesma pessoa, antes de comparar bebidas. O Prolog volta à escolha de amigo(Y) e tenta outro Y. As duas respostas finais são (ana,caio) e (caio,ana), pois ambos gostam de cafe.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro1 primeiro escolhe um homem e depois verifica que casado(X) não pode ser provado, retornando osvaldo, paulo e jake. solteiro2 tenta negar casado(X) enquanto X está livre: como existe algum casado, a negação falha. Osvaldo aparece como solteiro apenas porque não há o fato casado(osvaldo); ser pai não é prova de casamento.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

Na chamada maximo3(5,3,3), a cabeça da primeira cláusula maximo3(X,Y,X) não unifica com os argumentos (5,3,3), de modo que o corte nela nunca é executado; a segunda cláusula aceita 3. Uma correção é: maximo3(X,Y,M) :- X >= Y, !, M = X. maximo3(_,Y,Y). Assim, a condição e o corte vêm antes da unificação com o resultado.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim começa chamando a si mesmo sem reduzir o problema, causando recursão à esquerda e esgotamento da pilha na busca em profundidade. ancestral_bom tenta primeiro o caso base e, no caso recursivo, avança por um progenitor até uma geração posterior.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

São 24 colorações com 3 cores, 0 com 2 cores e 1728 com 4 cores. É declarativo porque o código estabelece as restrições (cores permitidas e vizinhos diferentes), enquanto o mecanismo de backtracking do Prolog realiza a

busca por combinações válidas.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma funcional, anexar recebe listas e devolve a lista concatenada. No lógico, concatena/3 descreve uma relação: além de juntar listas, pode descobrir seus componentes a partir do resultado, por unificação e backtracking.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

(1) Negação como falha em solteiro1/2: cadastro incompleto e ordem das metas podem levar a conclusões inadequadas. (2) Recursão à esquerda em ancestral_ruim/2: a estratégia de busca pode entrar em laço ou esgotar a pilha.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para sistemas de regras, inferência, relações familiares, configuração e problemas de restrições, como a coloração de mapas. Preferiria Java para aplicações corporativas gerais, interfaces, integrações e processamento de grande escala, em que seu ecossistema e controle operacional são mais adequados.